

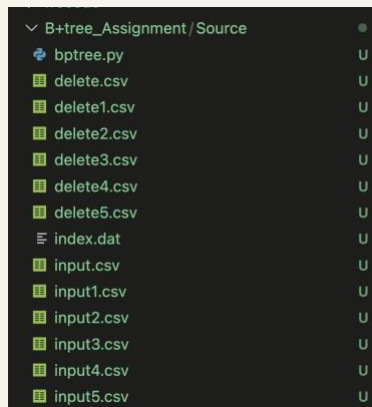
B+ Tree Assignment

2024084648 이종민

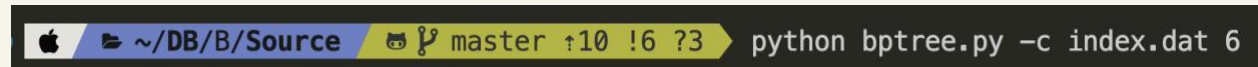
Instructions for compiling source codes at TA's computer

Python 3.10 이상 버전 필요 (match case문)

표준 라이브러리만 사용하므로 별도의 패키지 설치의 필요하지 않습니다.



1. bptree.py와 같은 폴더 위치에 insert, delete csv 파일을 저장합니다.
2. 해당 폴더 경로를 터미널에서 연 상태에서, 과제 명세의 명령어를 입력하면 됩니다.



(index.dat 파일은 같은 폴더 위치에 생성됩니다.)

명령어 형식

기능	명령어
index 파일 생성 (N : 최대 자식 수)	python bptree.py -c index.dat 6
삽입	python bptree.py -i index.dat input.csv
삭제	python bptree.py -d index.dat delete.csv
단일 키 검색	python bptree.py -s index.dat 125
범위 검색	python bptree.py -r index.dat 100 200

알고리즘 요약

index 파일에 노드를 저장하는 디스크 기반 B+ tree를 구현했다. 프로그램은 명령어마다 한 번 실행되며, 헤더에서 트리 정보를 읽어 작업을 수행한 뒤 변경 사항을 파일에 반영한다.

파일 구조 : index 파일은 32 bytes 헤더 (newOffset, freeHeadOffset, rootOffset, N)와 고정 크기의 노드들로 구성되며, 노드는 offset으로 접근한다.

변경 관리 : 작업 중 변경된 노드는 바로 파일에 쓰지 않고 modified에 모아 두었다가, 작업이 끝나면 patch()로 한 번에 반영한다.

삽입 : findLeaf()로 key가 들어갈 leaf를 찾아 정렬된 위치에 넣는다. 노드가 가득 차면 split()으로 나누고 분리 키를 부모로 올리며, 필요하면 루트까지 전파된다.

삭제 : leaf에서 key를 지운 뒤 최소 키 개수보다 적어지면 rebalance()에서 형제와 merge()하거나 distribute()로 키를 빌려온다. 이 과정도 위로 전파될 수 있다.

검색 : 단일 키 검색은 루트부터 leaf까지 내려가며 경로를 출력하고, 범위 검색은 시작 leaf에서 right sibling을 따라가며 키를 출력한다.

노드 재사용 : 삭제된 노드는 linked list로 연결해 두었다가 새 노드를 만들 때 먼저 재사용한다.

자료 구조

1. Node

B+tree에서 runtime에 노드 관리를 위한 class

m : 노드의 키의 개수를 저장

p : 리프인 경우 [key, value], 비리프인 경우 [key, left child] 쌍의 이중 리스트

r : 리프인 경우 right sibling node, 비리프인 경우 rightmost child의 포인터 (index 파일에서의 Node 주소), right sibling이 없으면 -1

isLeaf : leaf이면 1 아니면 0

offset : index 파일에서 파일 헤더로부터 노드의 상대 주소.

2. FreeNode

해제된 노드를 링크드 리스트 형태로 모으기 위한 class

offset : Node와 동일

nextOffset : 다음 노드의 offset을 저장함 말단 노드인 경우 -1

관련 함수

writeNode(index file, Node)

파일 포인터를 헤더 크기 + node.offset으로 옮긴 뒤 Node의 인스턴스 변수를 struct로 pack (isLeaf 제외하고 모두 signed long long, offset은 노드를 읽는 시점에 항상 알 수 있으므로 file에 적지 않음)해서 index 파일에 write한다.

readNode(offset) -> Node

Index 파일에서 헤더 크기 + offset만큼 파일 포인터를 옮긴 뒤, 노드 크기만큼 읽은 후 struct를 unpack하여 값을 받아 새로운 node 객체를 생성해 반환한다.

만약 해당 offset이 modified에 존재하는 경우 파일에서 읽어오는 대신 modified의 node를 그대로 반환한다.

writeFreeNode(index file, FreeNode)

파일 포인터를 옮긴 후, 다음 노드의 offset을 struct로 pack해 write한다. node와 마찬가지로 본인 offset은 적지 않는다.

readFreeNode(offset) -> FreeNode

해당 주소의 struct를 읽어 unpack해 FreeNode 객체에 담아 반환한다.

allocOffset() -> int

새로운 Node의 offset이 필요할 때 호출된다.

만약 FreeNode가 존재한다면 (FreeHeadOffset != -1) FreeNode 위치의 주소를 반환하고 FreeHeadOffset을 nextOffset으로 바꾸고

이외에는 newOffset을 반환하고 newOffset += NodeSize를 해 업데이트한다.

releaseOffset()

할당된 노드를 해제할 때 FreeNode linked list로 엮기 위해 사용한다.

freeNode가 없는 경우 새로운 FreeNode를 만들어 writeFreeNode()를 호출한다.

존재하는 경우 Linked list 꼬리까지 반복문을 통해 도달해, 꼬리 FreeNode 뒤에 붙여주고 두 freeNode 모두 writeFreeNode() 해준다.

B+Tree 함수

main 루틴

1) sys.argv 입력을 파싱해 create(), insert(), delete(), keySearch(), rangedSearch() 함수를 호출한다.

2) index 파일을 열고 헤더 부분을 읽어

newOffset : 8bytes signed long long, 새 노드를 만들 때의 offset

freeHeadOffset : 8bytes signed long long, FreeNode linked list의 head node의 offset

rootOffset : 8bytes signed long long, root node의 offset

N : 8bytes signed long long, 최대 자식 수

값을 변수에 저장하고, 이 값을 바탕으로 파일 헤더 크기, node size, node 포맷, freeNode 포맷을 초기화한다.

3) offset을 키로 가지고 [node, 상태('del', 'mod')]를 값으로 갖는 modified dictionary를 초기화하고 patch() 함수로 변경 사항 (modified)을 index 파일에 반영한다.

1. create()

Index 파일을 열어 초기 파일 헤더 값을 write한다.

2. insert(input.csv)

input.csv 파일을 열어 key, value를 한 줄씩 읽어

_insert() 함수 매개변수로 전달

1) _insert(key, value)

root가 없는 경우 : 새 root node를 만들고 rootOffset을 수정한 후 modified[]에 추가한다.

이외의 경우 readNode(rootOffset)로 루트 노드를 읽고 findLeaf(root, key, path)를 통해 해당 키를 가진 leaf 노드를 찾아 targetNode에 저장하고 경로를 path 리스트로 받는다.

반복문을 통해 targetNode.p를 순회하며 만약 key가 이미 존재한다면 변경 없이 반환하고 타겟보다 작은 키면 continue하고 타겟보다 크다면 그 위치에 insert하고 modified에 추가하며 노드 크기를 확인한다. 만약 N보다 target.m + 1이 크다면 split() 함수를 호출한다.

반복문에서 걸리지 않았다면 가장 오른쪽 키가 되는 것이므로 그곳에 추가하고 같은 과정을 진행한다.

(1) findLeaf(node, key, path = None, trace = None)

Node.p를 순회하며 key가 있는 노드로 재귀적으로 접근한다.

종료조건은 node가 leaf일 때다.

path가 입력된 경우 n을, trace가 입력된 경우는 n과 n.p의 index를 리스트로 저장한다.

(2) split(node, path)

node를 n, rightNode로 반씩 나눈다. path.pop()으로 부모를 찾아 rightNode의 0번째 키를 부모의 p에 알맞은 위치에 순회를 통해 넣는다. 만약 넣은 뒤 부모가 제한보다 많은 key를 갖는다면 split()한다. 따라서 split()이 재귀적으로 위로 전파될 수 있다.

루트인 경우 새 루트 노드를 만들고, 비리프 노드인 경우 중복을 없애기 위해 rightNode의 p[0]은 pop()한다.

3. delete(delete.csv)

delete.csv 파일을 열어 key를 한 줄씩 읽어

_delete() 함수 매개변수로 전달

1) _delete()

readNode(rootOffset)로 읽은 루트에서 findLeaf(root, key, trace)로 key가 있는 leaf 노드와 경로 trace를 받는다.

leaf.p에서 key를 찾아 삭제하고 leaf.m을 1 줄인다. key가 없다면 변경 없이 반환한다.

leaf를 modified에 추가한 뒤 rebalance(leaf, trace)를 호출한다.

(1) rebalance(node, trace)

node가 루트인 경우 (trace가 비어 있으면) 그대로 반환한다.

키 개수가 최소 키 개수보다 적으면 trace.pop()으로 부모와 인덱스 i를 받아 양쪽 형제 노드를 읽는다.

형제와 합쳐도 N을 넘지 않으면 merge()하고, 불가능하면 distribute()한다. 이후 부모가 최소 조건보다 작아지면 rebalance()를 호출하므로 재귀적으로 위로 전파될 수 있다.

a) merge(parent, left, right, i, rightmost)

right 노드를 left 노드에 합친다. 리프인 경우 right.p를 이어 붙이고 left.r = right.r로 연결을 유지하며, 비리프인 경우 부모의 분리 키를 내려 함께 붙인다.

부모에서 분리 키를 삭제하고 left를 가리키도록 포인터를 수정한 뒤, right는 'del'로 modified에 추가한다.

루트의 키가 0개가 되면 합쳐진 노드가 새 루트가 된다.

b) distribute(parent, left, right, n, i)

키를 하나 내줘도 최소 조건을 만족하는 형제 (왼쪽 우선)에게서 키를 하나 빌려온다.

리프인 경우 키를 옮긴 뒤 부모의 분리 키를 갱신하고, 비리프인 경우 부모의 분리 키를 내리고 형제의 키를 부모로 올린다 (부모를 통한 회전).

변경된 parent, 형제, n을 modified에 추가한다.

4. keySearch(key)

루트부터 호출한 findLeaf()를 통해 키가 있는 node와 경로 path를 받는다. 해당 노드에 search()를 호출해 value를 찾는다. 경로와 value를 출력한다. value가 없다면 NOT FOUND를 출력한다.

루트가 없는 경우 NOT FOUND 출력 후 반환한다.

1) `search(node, key) -> int`

node.p를 순회해 key가 있으면 value를 반환하고 없으면 None을 반환한다.

5. `rangedSearch(start, end)`

루트부터 `findLeaf(root, start, path)`하여서 start 키가 있을 수 있는 leaf node를 찾는다.

해당 리프 노드.p에서 순회를 시작해 start보다 크거나 같은 키들을 출력한다. 순회의 끝에 도달하면 leaf.r 노드로 넘어가서 다시 순회한다.

만약 end보다 키가 크거나 leaf.r이 없다면 (-1이라면) 함수가 종료된다.

6. `patch(modified)`

변경사항을 파일에 적용하기 위해

`patchNode()`, `patchHeader()`를 호출한다.

1) `patchNode(index file, modified)`

modified의 키를 순회하며 상태가 'del'이면 `releaseOffset()`, 'mod'이면 `writeNode()`를 호출한다. 따라서 파일에 노드들이 업데이트된다.

2) `patchHeader(index file)`

헤더 정보를 파일의 0번 주소부터 struct로 pack하여 적는다.

구현 특이사항

노드 크기 : m (8 bytes), isLeaf (1 byte), r (8 bytes)와 N - 1개의 [key, value 또는 child] 쌍 (각 16 bytes)으로, $17 + (N - 1) \times 16$ bytes이다.

파일 쓰기 최소화 : modified를 사용하므로 한 번 실행하는 동안 같은 노드를 여러 번 수정해도 파일에는 한 번만 쓴다.

Split 비율 : 노드가 넘치면 ($m = N$) $mid = m // 2$ 를 기준으로 나누며, 왼쪽 노드가 더 작거나 같게 나뉜다. 리프 노드는 왼쪽 mid개, 오른쪽 $m - mid$ 개로 나눈다 (예: 키 5개 $\rightarrow 2 : 3$, 4개 $\rightarrow 2 : 2$). 비리프 노드는 오른쪽의 첫 키를 부모로 올리므로 왼쪽 mid개, 오른쪽 $m - mid - 1$ 개가 된다 (예: 키 5개 $\rightarrow 2 : 2$, 4개 $\rightarrow 2 : 1$).

Free list : 삭제된 노드는 FreeNode로 바뀌 linked list로 연결하고, 헤더의 freeHeadOffset이 그 head를 가리킨다. 해제할 때는 리스트의 꼬리에 붙이고 새 노드가 필요할 때는 head부터 꺼내 재

사용하므로, 삭제와 삽입이 반복되어도 파일 크기가 불필요하게 커지지 않는다.

Test

구현 검증을 위해 별도로 작성한 테스트 스크립트 test.py의 기능은 다음과 같다. 각 테스트는 실행 결과를 정답과 비교하고, index 파일의 트리 구조가 B+ tree 조건을 만족하는지 검사한다.

테스트 종류

테스트	내용
insert	삽입 결과 검증
duplicate	중복 키 삽입 시 트리가 변하지 않는지 확인
delete	삭제 결과 검증
deleteall	전체 삭제 후 빈 트리 확인, 재삽입 시 해제된 노드 재사용 확인
range	여러 구간의 범위 검색 결과 확인
fuzz	무작위 삽입·삭제를 반복하며 검증

결과

```
PASS deleteallTest(N=7, input3)
PASS deleteallTest(N=7, input4)
PASS deleteallTest(N=7, input5)
PASS rangeTest(N=7, input)
PASS rangeTest(N=7, input1)
PASS rangeTest(N=7, input2)
PASS rangeTest(N=7, input3)
PASS rangeTest(N=7, input4)
PASS rangeTest(N=7, input5)
PASS fuzzTest(N=7, seed=1)
PASS fuzzTest(N=7, seed=2)
PASS fuzzTest(N=7, seed=3)

132 passed, 0 failed
```

The image shows a terminal window with a dark background. At the bottom, there is a status bar with icons for Apple, a folder, the path ~/DB/B/Source, a GitHub icon, the text 'master +13 !6 ?18', and a green arrow icon.